A SESSION ON MODERN REACT ARCHITECTURE

Local-First React

Building Apps That Don't Care If the Network Is Down

Debajit Mallick

Software Engineer @Ergeon | Organizer @GDG Siliguri & @React Siliguri

Who am I?

Debajit Mallick

Software Engineer

@Ergeon

Community Organizer

GDG Siliguri
React Siliguri

36+

talks delivered

LinkedIn

2024 Top Voice

Web Development

36+ talks delivered

Web Performance, Frontend, Firebase & more

Featured on Google For Developers

India YouTube — community contribution

SIH Winner

Team Delenitors (2020) — Software Edition

Mentor — SIH 2022 Winners

Team OrganiCod3rs, Software Edition

Mentor & Judge — Hack4Bengal

3.0 and 2.0 editions

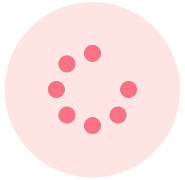
Top Contributor — GWoC & JWoC

GirlScript Winter of Code '21, JGEC '20

THE PAIN

We've been building React apps the same way for a decade

Render a spinner. Fetch from an API. Handle the loading state. Pray the network holds.



Loading spinners everywhere

Every component waits for the network. Users stare at skeletons before they can do anything.

Offline = broken

Drop the connection on a flight, in the metro, in rural area — the app stops working.

Optimistic-update bugs

Cache invalidation, stale data, rollback logic. We're shipping bugs that shouldn't exist.

THE IDEA

What if the client was the database?

Local-First

Your data lives in the client.

It syncs in the background.

Offline by default.

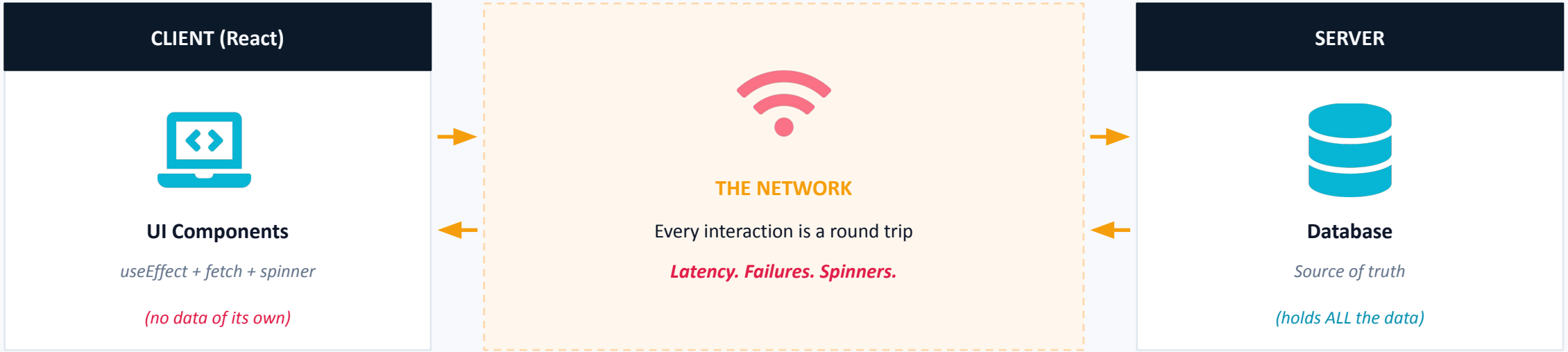
Conflicts handled for you.

The Seven Ideals

(Ink & Switch, 2019)

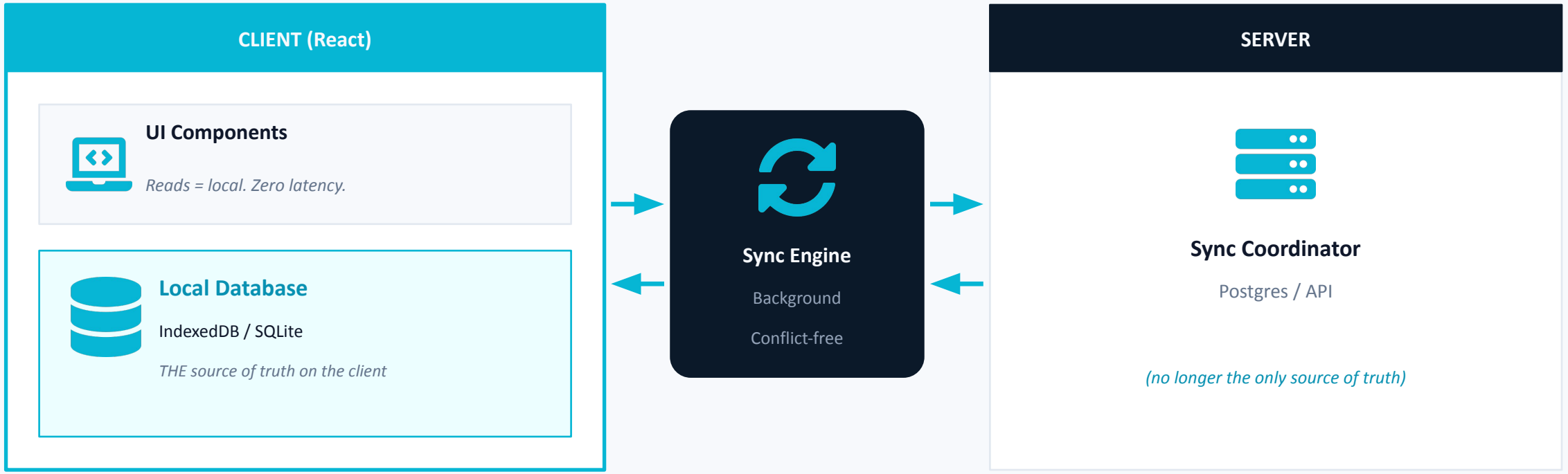
- No spinners — instant local reads
- Works offline — by default, not as an edge case
- Multi-device — sync across your devices
- Real-time collaboration — built in (Conflict-free Replicated Data Types or CRDTs)
- Long-lived data — files outlive the service
- Privacy & security — data stays with the user
- User ownership — you own your data

The traditional cloud-first app



The client is a temporary view. Kill the network, and your app dies.

The local-first app



The client owns the data. The network becomes an optimization, not a dependency.

THE LANDSCAPE

Three tools leading the local-first wave in 2026

Zero

by Rocicorp

Best DX, end-to-end

WHAT IT IS

Successor to Replicache. Ships its own query language (ZQL), its own sync server (zero-cache), and protocol.

HOW IT WORKS

Server-authoritative writes. Reactive query subscriptions. Rebase-on-conflict handling for true sync semantics.

Best for: Greenfield collaborative apps with Postgres

ElectricSQL

by Electric

Postgres-native sync

WHAT IT IS

Syncs your real Postgres database to client-side SQLite (via PGlite or WASM). Declarative 'shapes' control what each client gets.

HOW IT WORKS

Bi-directional active-active replication. Last-write-wins by default. React hooks like useShape stream live updates.

Best for: Apps where Postgres is already the backend

TinyBase

by James Pearce

Tiny, zero deps

WHAT IT IS

A reactive in-memory data store with built-in indexing, queries, CRDT sync, and React bindings — all in ~6 to 14 kB.

HOW IT WORKS

Pluggable persisters (IndexedDB, SQLite, PGlite). Native CRDT support. Pair with any sync layer or stand alone.

Best for: Lightweight apps, prototypes, mobile-first

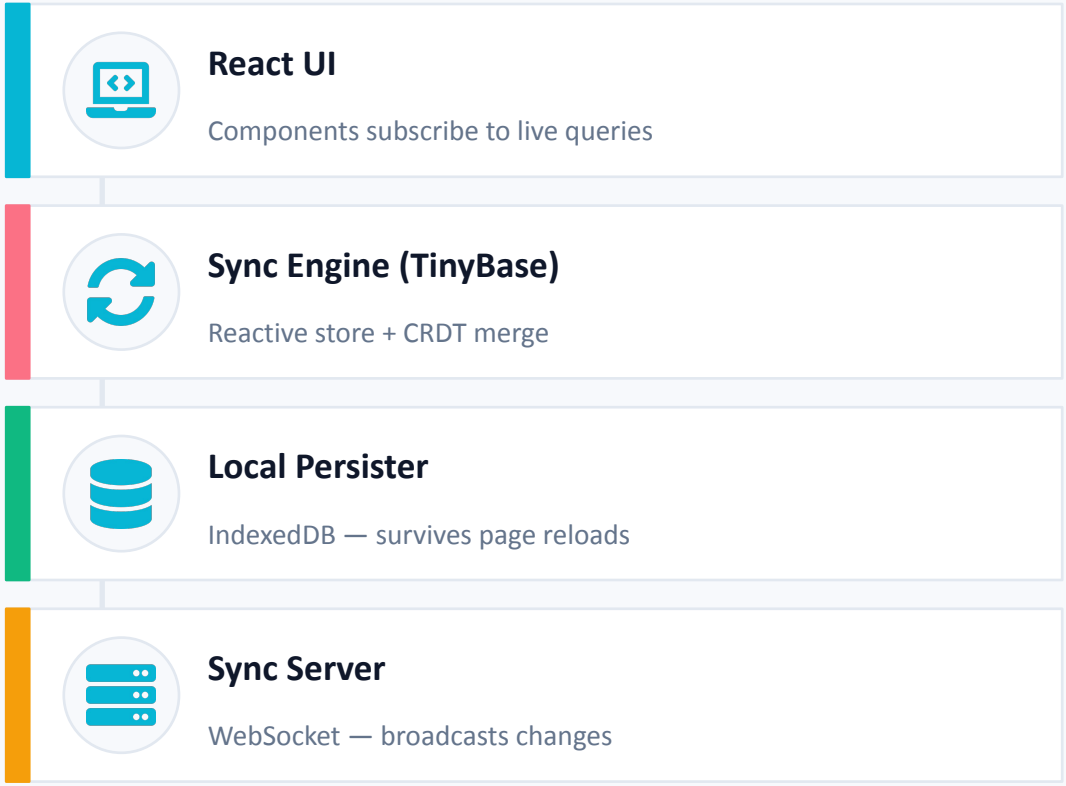
LIVE DEMO

Let's build a todo app

that doesn't care if the WiFi dies

Open two tabs · Edit offline · Watch them sync

How we're going to wire this up



What we'll show

1. **Spin up a React + Vite app**
   Plain useState first — to set the baseline
2. **Drop in TinyBase**
   Replace state with a reactive local store
3. **Add IndexedDB persistence**
   Refresh the page — data survives
4. **Kill the network in DevTools**
   App keeps working. No errors, no spinners
5. **Open two tabs · sync**
   Bring network back. Both tabs converge

Switching to VS Code

Let's write some code

GOTCHAS

The things that bite you in production

Auth & permissions are harder

If data lives on the client, 'who can read what' is no longer a server-side query. You need sync rules, row-level permissions, or a server-authoritative model.

Schema migrations get tricky

Client has v1 data. Server is on v3. The user opens the app after 6 months. Plan for forward and backward compatibility from day one.

Partial sync is a design problem

You can't sync the whole DB to every client. 'Shapes' or queries define each user's slice. Get this wrong and you sync gigabytes — or miss data.

Bundle size is real

Local DB + sync engine + CRDT logic = heavier bundle. Worth it for the right app, painful for a marketing site. Pick the right tool for the job.

DECISION GUIDE

When local-first is the right call

REACH FOR IT WHEN…

- ✓ Collaboration is core — multiple users on the same data
- ✓ Offline is a real requirement, not a 'nice to have'
- ✓ You want instant UI — Linear, Figma, Notion-level snappiness
- ✓ Users edit the same record from multiple devices
- ✓ Data privacy / ownership matters to your users

DON'T BOTHER WHEN…

- ✗ You're building a content site / blog / landing page
- ✗ Data is server-computed (search, analytics, dashboards)
- ✗ You need strong consistency — banking, inventory, ticketing
- ✗ The dataset is too big to fit on the client
- ✗ Your team can't take on the new mental model right now

WRAPPING UP

Three things to take home

01

The client can be the database

Local-first inverts the cloud-first model. Reads are instant, writes are local, sync is a background detail.

02

The tooling is finally here

Zero, ElectricSQL, TinyBase, TanStack DB, Replicache — pick one based on your backend and your scale.

03

Try it on something small first

Don't migrate your monolith. Build one feature local-first — a sidebar, a draft editor, a settings panel.

Q&A

Let's chat — connect with me here

LinkedIn

/in/debajit-mallick/

Twitter / X

@MallickDebajit

GitHub

@debajit13

GOING DEEPER

Resources to keep learning

Foundational reading

- The Local-First Manifesto — Ink & Switch (2019)
- CRDTs: The Hard Parts — Martin Kleppmann (YouTube)
- localfirstweb.dev — the community hub

Sync engines

- zero.rocicorp.dev — Zero by Rocicorp
- electric-sql.com — ElectricSQL + PGlite
- tinybase.org — TinyBase docs and demos

Adjacent worth knowing

- tanstack.com/db — TanStack DB
- yjs.dev / automerge.org — CRDT libraries
- instantdb.com / convex.dev — hosted local-first

Thank you

Now go build something that doesn't need the network.

Debajit Mallick

Software Engineer @Ergeon | GDG Siliguri | React Siliguri